

Sorting Algorithm Benchmark Report

Assignment 1 — Algorithms & Java Benchmarking

1. Introduction

This assignment benchmarks different sorting algorithms in Java. **Bubble Sort**, **Quick Sort**, **LSD Radix Sort**, and Java's built-in **Arrays.sort()** were tested on arrays of 50k and 1 million integers with different input distributions (random, sorted, reverse, nearly sorted). Execution time was measured using JMH and results compared.

2. Setup

- JDK 21.0.8, JMH 1.37
- Warmup: 3 iterations, Measurement: 5 iterations, Fork: 1
- Bubble Sort: $N = 50,000$ ($O(n^2)$ makes $N = 1,000,000$ infeasible)
- All other algorithms: $N = 1,000,000$

3. Implementation

Each algorithm was implemented in a separate Java class.

- **Bubble Sort** — classic version with early-exit optimization (tested only on $N = 50,000$ due to $O(n^2)$ complexity).
- **Quick Sort** — in-place, uses a random pivot to prevent $O(n^2)$ behavior on sorted/reverse data.
- **LSD Radix Sort** — 4 passes, base 256, supports negative numbers.
- **Arrays.sort()** — Java's built-in reference sorter.

Arrays were generated for uniform random, sorted ascending, reverse sorted, and nearly sorted distributions. Correctness was verified using SortVerifier.

4. Results

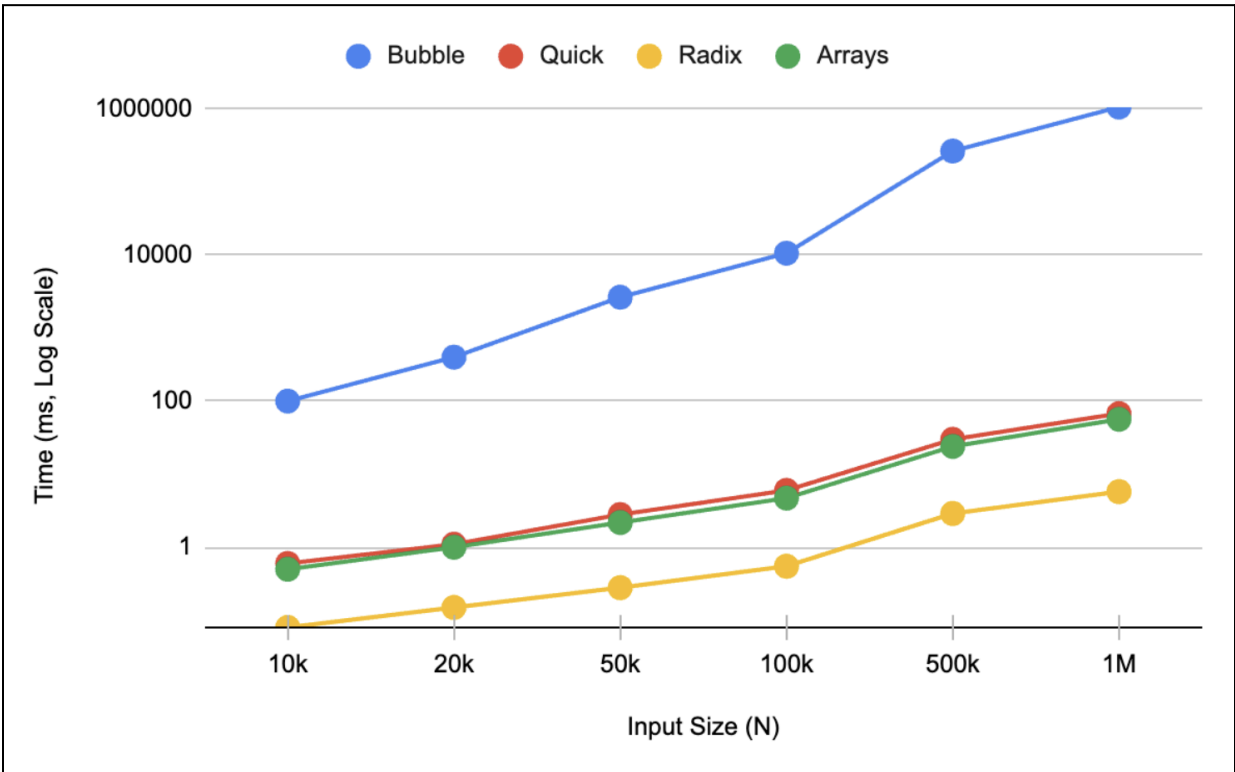
Table 1 — All algorithms, $N = 50,000$ (ms/op)

Algorithm	Random	Sorted	Reverse	Nearly Sorted
Bubble Sort	2646.839	0.014	473.688	455.007
Quick Sort	2.800	1.132	1.275	1.368
LSD Radix Sort	0.281	0.602	0.614	0.611
Arrays.sort	2.186	0.015	0.035	0.656

Table 2 — Fast algorithms, N = 1,000,000 (ms/op)

Algorithm	Random	Sorted	Reverse	Nearly Sorted
Quick Sort	67.555	24.521	27.636	29.585
LSD Radix Sort	5.774	15.112	15.073	14.870
Arrays.sort	56.442	0.284	0.735	17.196

Figure 1: Execution time comparison of sorting algorithms for different input sizes.



Note: Bubble Sort was only benchmarked at N = 50,000. Larger input sizes are estimated based on its $O(n^2)$ complexity. Bubble Sort was excluded from the 1M test based on the 50k results — extrapolating its $O(n^2)$ growth gives an estimated runtime of ~17 hours, making it impractical to benchmark.

5. Discussion

Bubble Sort becomes impractical for large datasets due to its $O(n^2)$ time complexity, completing 50k elements in over 2.6 seconds. Quick Sort scales well with $O(n \log n)$ performance, and the random pivot prevents stack overflow on sorted inputs. LSD Radix Sort is the fastest overall for large datasets, operating in linear time for fixed-length integers and performing consistently across all input distributions. Arrays.sort() is highly efficient, especially on sorted or nearly sorted data, and provides a reliable reference. The chart highlights the large performance gap between Bubble Sort and the other algorithms.